Experiment 1 — The ‘Compile[]‘ engine

Mathilda — execution-speed experiments

Contents

Experiment 1 — The **`Compile[]`** engine 1
- Hypothesis . . . 1
- What was built . . . 1
- Results . . . 2
- What the subset costs when you fall out of it . . . 2
- Verification . . . 3
- Still open . . . 3
- Why Mathilda is not the fastest here, and what it would take . . . 3

Experiment 1 — The **`Compile[]`** engine

Dates: 2026-07-26 → 2026-07-28 · Commits: `48ab095` (M0) … `75e8d8a` (v0.019) · Code: `src/compile/` · Design: `docs/design/compile.md` · Result: ~234× over the interpreter; 1.4–2.4× faster than Mathematica's **`Compile[]`** on every scalar kernel measured

Common method in `README.md`.

Hypothesis

Mathilda's evaluator is a tree walker with infinite-evaluation semantics: every arithmetic node allocates, every symbol lookup hashes, every step re-checks attributes and re-tries `DownValues`. That is the right architecture for symbolic work and the wrong one for a loop that adds two doubles ten million times.

Mathematica's answer to the same problem is `Compile[]`, and the hypothesis was that Mathilda needed the same thing for the same reason — not as a bolt-on, but as a shared engine that the numeric builtins could also reach (experiment 4).

The prior art was already in the tree: NDSolve had its own RHS compiler (`project_ndsolve_rhs_compiler`, >800× on stiff systems). It worked, it was narrow, and it proved the approach. M0 generalised it.

What was built

A typed, scalar, register-machine bytecode VM.

- Types (`CompileType`): `_Integer`, `_Real`, `_Complex`, and `_Real`/`_Integer` arrays of any rank. Types are inferred where possible and declared where not.
- IR and bytecode: a flat instruction stream, pc-based, with a small register file. No tree walking at run time.
- Dispatch: computed-goto (threaded) rather than a `switch`, so the branch predictor sees one indirect jump per opcode with a stable history, instead of one heavily-mispredicted N-way branch.
- Kernels: one shared machine-precision registry (`src/ndkernels.c`) behind a generic `KERNEL` opcode, so a special function is one table entry rather than one opcode. 103 numeric heads; 93 covered.

- Control flow, designed in from the start rather than retrofitted: `If`/`Which` branches (M2a), `Sum`/`Product` counted loops (M2b), `Do`/`While`/`For`/`Nest` (M2c), and a dedicated `OP_LOOP` close for counted loops.
- A user-facing surface: `Compile[]`, a `CompiledFunction` object, `CompilePrint` (bytecode disassembly), `CompileDiagnostics`, and `RuntimeAttributes -> Listable`.
- **`MainEvaluate`** fallback: anything outside the subset calls back into the interpreter rather than failing to compile.

Results

Against the interpreter (same binary, same source)

kernel	interpreted	compiled	
Wave-equation stencil (`COMPILE_EXAMPLE.md`)			~234×
Complex Newton fractal, 200×200×25	4.45 s	0.046 s	97×
`Nest[(u + 2./u)/2 &, 3., 2·10⁶]`	0.081 s	0.032 s	2.5× (over `numloop`)
`While`, 10^6 iterations	0.023 s	0.015 s	1.6× (over `numloop`)

The last two rows compare `Compile[]` against `numloop` (`AUTO_COMPILATION.md`), not against the tree walker — by the time this was measured the interpreter already auto-compiled those bodies, so the tree-walking number is no longer reachable from ordinary code.

Against Mathematica's **`Compile[]`** and against Python

Same source text in both CAS. The Python column is the honest Python answer for a scalar loop of this shape, which is CPython — numba is not installed on this host, so this is not a JIT comparison; it is what a Python user without one would measure.

Benchmark	Mathilda	Mathematica 14.0	NumPy / Python	vs WL	vs NumPy
Logistic map, 10^7 iterations	183.76 ms	254.36 ms	460.57 ms	1.38x	2.51x
Mandelbrot, 800x800, 100 iterations	769.23 ms	1.621 s	980.20 ms	2.11x	1.27x
Lennard-Jones energy, 1452 bodies (all pairs)	133.19 ms	319.51 ms	91.25 ms	2.40x	1/1.46x
Monte Carlo pi, 10^7 samples (vectorized)	162.36 ms	250.40 ms	204.53 ms	1.54x	1.26x

The Python column is vectorised NumPy for Lennard-Jones, Mandelbrot and Monte Carlo π — so those three are genuine library comparisons — and a CPython loop for the logistic map, which is a scalar recurrence with no vectorised form (numba is not installed on this host). Only the logistic-map row should be read as "what a Python user without a JIT measures".

Mathilda's compiled scalar code is faster than Wolfram's on every compiled kernel measured, by 1.4–2.4×. The three bodies are deliberately unalike: a nested all-pairs `For` loop with indexed array reads (Lennard-Jones), an escape-time loop with a compound && guard (Mandelbrot), and a bare arithmetic recurrence (the logistic map). The same result appears independently in `COMPILE_EXAMPLE.md` for a wave-equation stencil, at 1.8–2.1×.

What the subset costs when you fall out of it

The most important finding of this experiment is not a speed number.

The compilable subset is a cliff, not a slope. A body that compiles runs at machine speed; a body that misses by one unsupported head falls all the way back to the interpreter — 10–40× — and, as originally built, did so silently. There was no way to tell a fast `Compile[]` from a slow one except by timing it.

That is why `CompileDiagnostics` and `CompilePrint` exist (M6, `6fd97ff`, `7740825`). A bail is now reportable, and the coverage audit that followed took the numeric-head count from 64 to 93 of 103 — driven by what the

audit found missing, not by what seemed likely.

The general lesson, recorded in `tasks/lessons.md`: a fast path with a silent fallback is an unmeasurable fast path. Every subsequent subsystem in these experiments (packed arrays, auto-compilation, the ND kernels) was given a diagnostic switch before it was given a fast path.

Verification

- `tests/test_compile.c` compares every compiled body against the interpreter on random inputs — parity by numeric distance where the result is a Real, and by structural equality where the result's head matters (see experiment 5).
- Result-head parity is its own test class: `Sign`, `IntegerPart`, `Quotient` and `FractionalPart` must return the same type compiled as interpreted, not merely the same value.
- One regression found this way, on 2026-07-31: a change to the scan path made `FoldList[Plus, 0., NDArray[…]]` answer with head `NDArray` while the equivalent `FoldList[Function[{p,q}, p+q], …]` answered with head `List`. Two spellings of one operation disagreeing is exactly what a cross-spelling parity test is for.

Still open

- 10 of 103 numeric heads remain uncovered.
- Complex arguments are the largest remaining result-head gap (`project_compile_result_head_parity`).
- Machine-integer overflow falls back to the interpreter rather than promoting in place — see MACHINE_INTEGERS.md.

Why Mathilda is not the fastest here, and what it would take

Three of the four kernels are ahead of both systems — 1.38–2.40× Mathematica and 1.26–2.51× Python. One is not:

row	Mathilda	best other	gap	cause
Lennard-Jones energy, 1452 bodies	133 ms	91.3 ms (NumPy)	1.46×	the all-pairs sum is a scalar loop here and a ve

The comparison is not like for like, and saying so is the point. The compiled CAS form is a doubly-nested scalar loop over 10^6 pairs; the NumPy form is an `Outer`-style whole-array expression that hands the same arithmetic to vectorised kernels. Both are the idiomatic way to write it in their own language.

The road to fastest

1. Vectorise the VM's inner loop. The bytecode interpreter executes one instruction per scalar operation. For a straight-line body with no data-dependent control flow — which the LJ kernel is — the same bytecode could be executed over a block of iterations at a time, which is what experiment 2's fused array path already does for expressions written over arrays. Expected: the gap closes entirely, since the arithmetic is identical.
2. Recognise the reduction. `e = e + f(i, j)` over a rectangular index space is a reduction with no loop-carried dependence other than the accumulator, so it can be strip-mined into partial sums and threaded. Mathilda's threaded fused map (experiment 2) already does this shape for array expressions and does not yet do it for `While`/`Do` loops.

Both are extensions of machinery that exists, and neither changes what the VM is. Against Mathematica's `Compile[]` the row is already 2.40× ahead.